

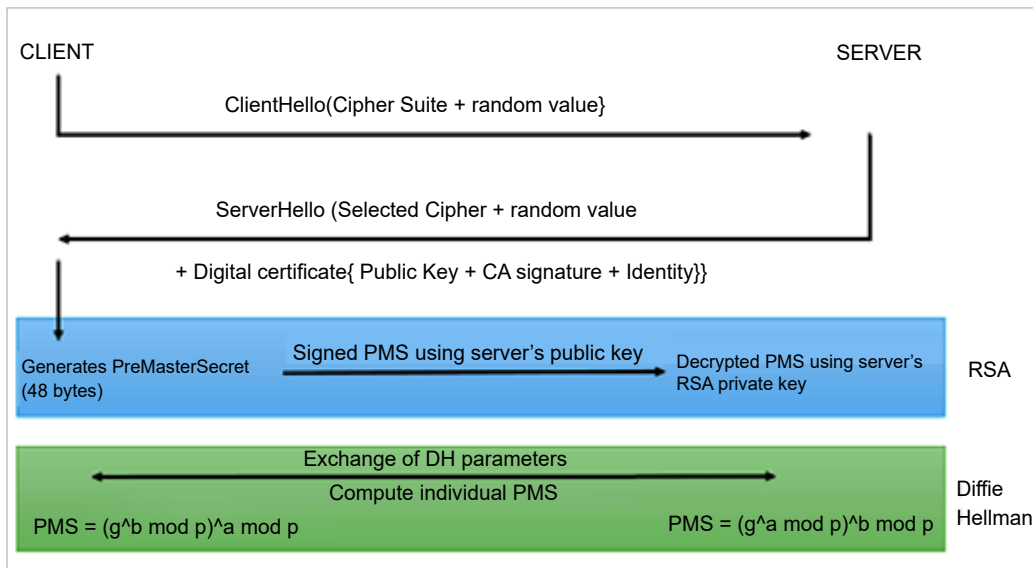


Figure 1: Mechanism of SSL 2.0 handshake protocol

Hellman Ephemeral (ECDHE) key exchange methods provide forward secrecy because they use ephemeral (temporary) key pairs for each session. Even if the server's private key is compromised, past session keys cannot be derived, as the ephemeral keys are discarded after the session ends. Thus, the attacker may manage to intercept future sessions but not any of the

previous or current sessions.

- **Step 2: Encryption with server's public key:** The client encrypts this PreMasterSecret using the server's public key, which the client obtained from the server's digital certificate sent during the ServerHello.
- **Step 3: Client sends the encrypted PreMasterSecret:** The encrypted PreMasterSecret is sent to the server as part of the ClientKeyExchange message.
- **Step 4: Server decrypts the PreMasterSecret:** The server decrypts the PreMasterSecret using its private key.

- Server computes: $PMS = (g^a \bmod p)^b \bmod p$

Both computations result in the same shared PreMasterSecret, but it's never transmitted over the network, providing security even if the session is intercepted.

Forward secrecy

Both RSA and Diffie-Hellman (DH) have their strengths and weaknesses in the context of SSL/TLS, and the choice between them depends on the specific requirements and threats. However, the method following Diffie Hellman is more secure in terms of forward secrecy, which means that even if the server's private key is compromised in the future, past communications cannot be decrypted. RSA key exchange does not provide this because the same private key is used across multiple sessions. If an attacker later obtains the server's private RSA key, they can decrypt any previously recorded communications that used RSA for the key exchange. This is because PreMasterSecret is encrypted directly with the server's public key, and the security of the session relies on the secrecy of the server's private key.

The Diffie-Hellman Ephemeral (DHE) or Elliptic-Curve Diffie-

Performance

RSA operations, particularly decryption on the server side, are computationally intensive but still faster than traditional Diffie-Hellman. Given that session keys are generated for each session, handling 1,000 sessions per minute could strain resources with Diffie-Hellman due to the overhead of key generation. In such high-load scenarios, RSA, even with its larger key sizes (e.g., 2048-bit keys), may prove more efficient and practical. The efficiency of RSA in these cases makes it a more suitable choice when balancing performance and security in environments with heavy traffic. Elliptic-Curve Diffie-Hellman (ECDH), however, is significantly faster and requires smaller key sizes for equivalent security, making it more efficient than RSA and traditional DH.

Hashing

In SSL 2.0, MD5 is used both during the handshake and after the connection is established.

Session key derivation: MD5 is used in combination with other hashing algorithms to help derive session keys. For example, MD5 is used along with

Diffie-Hellman (DH) key exchange

- **Step 1: Client and server exchange DH parameters:** Both the client and the server exchange Diffie-Hellman parameters (g , p , and public values $g^a \bmod p$ and $g^b \bmod p$).
- **Step 2: Client and server independently compute the PreMasterSecret:** Both the client and server compute the PreMasterSecret independently using their own private key and the other party's public key. For example:
 - Client computes: $PMS = (g^b \bmod p)^a \bmod p$